

Embree 到 cuBQL GPU 加速迁移可行性报告

项目: STARDIS-GPU GPU 加速辐射传输求解器

日期: 2026-01-23

状态: ✔ 技术可行性已确认, 建议实施

执行摘要

问题背景

性能剖析数据显示, **CPU 执行时间的 50% 消耗在 Embree 模块**, 主要集中在射线-BVH 相交查询的单一入口点。这一瓶颈为 GPU 加速提供了明确的优化目标。

解决方案

将射线追踪从 Intel Embree (CPU) 迁移至 NVIDIA cuBQL (GPU), 利用 RTX GPU 的大规模并行能力。

核心结论

- ✔ **技术可行性:** 已确认 - cuBQL 支持所需的全部核心功能
- ✔ **依赖隔离:** 已验证 - Embree 依赖完全封装在 star-3d 库内
- ✔ **API 兼容:** 可实现 - 通过适配层保持公共 API 兼容
- ✔ **性能预期:** 保守估计整体应用加速 **1.8-2.0x** (受 Amdahl 定律限制)

迁移范围

- 影响范围:** 仅 star-3d 库 (约 2000 行代码)
- 不影响:** 上游模块 (stardis-solver, star-enclosures-3d 等) 无需修改
- 核心目标:** BVH 构建 + 射线相交查询

一、技术背景分析

1.1 依赖范围确认

依赖泄露分析结果

关键发现: Embree 类型通过 star-3d 的**内部中间层头文件**暴露 (非直接通过公共 s3d.h):

内部头文件	暴露的 Embree 类型	用途
s3d_device_c.h	RTCDevice rtc;	设备管理
s3d_geometry.h	RTCGeometry rtc;	几何数据

```
s3d_scene_view.c.h    RTCScene rtc_scn;    场景管理
                      RTCGeometry rtc_geom;
```

暴露路径:

```
s3d.h (公共 API)
├─ s3d_backend.h      → #include <embree4/rtcore.h> (直接引用)
├─ s3d_geometry.h     → RTCGeometry 结构体字段
└─ s3d_scene_view.c.h → RTCScene 结构体字段
```

外部访问验证

重要结论: ☒ 外部模块不访问暴露的 Embree 类型

经过全面搜索验证:

- ☒ 零次外部包含内部头文件 (s3d_geometry.h 等)
- ☒ 零次外部使用 Embree 类型 (RTCDevice, RTCScene 等)
- ☒ 外部代码仅使用 s3d.h 公共 API

安全性评估:

- 迁移隔离度: 高 - Embree 替换不会影响外部依赖
- 重编译级联: 无 - 外部模块无需修改
- 风险等级: 低

1.2 性能瓶颈定位

剖析数据:

- 50% 时间消耗在 Embree 模块
- 单一热点: rtcIntersect1() 射线相交查询
- 调用频率: 每帧数百万次射线查询

Amdahl 定律约束:

如果射线追踪占运行时间的 50%
且 GPU 对该部分提供 50x 加速
则整体加速比 = $1 / (0.5 + 0.5/50) \approx 1.98x$

即使 GPU 加速达到无限快 (∞x)
整体最大加速比也仅为 2.0x

保守性能预期:

组件	Embree (CPU)	cuBQL (GPU)	加速比
BVH 构建	100 ms	20 ms	5x
100 万射线相交	500 ms	10 ms	50x
CPU-GPU 传输开销	0 ms	20 ms	N/A
总计 (含开销)	600 ms	50 ms	12x
应用整体	-	-	~2x

二、API 迁移映射方案

2.1 核心 API 映射表

设备管理

Embree API	cuBQL 等效方案	说明
<code>rtcNewDevice(config)</code>	无需创建 - 使用当前 CUDA 上下文	cuBQL 操作在当前 CUDA 设备上
<code>rtcSetDeviceErrorFunction()</code>	<code>cudaGetLastError()</code> + 宏封装	使用 CUDA 错误处理模型
<code>rtcReleaseDevice()</code>	无需释放	由 CUDA 上下文管理

场景/BVH 创建

Embree API	cuBQL 等效方案	说明
<code>RTCScene rtcNewScene()</code>	<code>cuBQL::BinaryBVH<float, 3> bvh;</code>	BVH 即场景结构
<code>rtcSetSceneBuildQuality(scene, quality)</code>	<code>BuildConfig cfg;</code> <code>cfg.makeLeaves = SAH_BASED;</code>	通过配置结构设置质量
<code>rtcCommitScene(scene)</code>	<code>cuBQL::gpuBuilder(bvh, boxes, count, cfg)</code>	核心瓶颈点 - GPU 构建 BVH
<code>rtcReleaseScene()</code>	<code>cuBQL::free(bvh)</code>	显式释放 BVH 资源

几何设置

Embree API	cuBQL 等效方案	说明
------------	------------	----

<code>rtcNewGeometry(device, TRIANGLE)</code>	计算三角形包围盒数组 <code>box3f[]</code>	cuBQL 使用预计算的边界框
<code>rtcSetSharedGeometryBuffer()</code>	<code>cudaMalloc()</code> + <code>cudaMemcpy()</code>	关键差异: 无零拷贝，需显式上传
<code>rtcCommitGeometry()</code>	无需提交 - 边界框数据已就绪	
<code>rtcAttachGeometry(scene, geom)</code>	将边界框包含在 <code>boxes[]</code> 数组中	所有几何数据传递给 <code>gpuBuilder()</code>

射线相交查询（关键瓶颈）

Embree API	cuBQL 等效方案	说明
<code>rtcIntersect1(scene, &rayhit)</code>	批处理 GPU 内核 + <code>shrinkingRadiusQuery::forEachPrim()</code>	设计模式变化: 单射线 → 批量射线
RTCRayHit 结构	分离的 <code>float3 org/dir</code> + <code>HitResult</code>	分离射线参数和结果
CPU 循环查询	GPU 并行内核	根本架构变化

2.2 关键代码转换模式

模式 A: 设备初始化

```
// ===== EMBREE (CPU) =====
RTCDevice device = rtcNewDevice(NULL);
rtcSetDeviceErrorFunction(device, errorCallback, userData);

// ===== cuBQL (GPU) =====
// 无需创建设备对象 - 确保 CUDA 上下文已初始化
cudaSetDevice(0); // 选择 GPU
// 使用 CUDA_CHECK 宏处理错误
#define CUDA_CHECK(call) do { \
    cudaError_t err = call; \
    if (err != cudaSuccess) { \
        fprintf(stderr, "CUDA Error: %s\n", cudaGetErrorString(err)); \
        exit(1); \
    } \
}
```

```
    } \
} while(0)
```

模式 B: BVH 构建

```
// ===== EMBREE (CPU) =====
RTCScene scene = rtcNewScene(device);
rtcSetSceneBuildQuality(scene, RTC_BUILD_QUALITY_MEDIUM);

RTCGeometry geom = rtcNewGeometry(device, RTC_GEOMETRY_TYPE_TRIANGLE);
rtcSetSharedGeometryBuffer(geom, RTC_BUFFER_TYPE_VERTEX, ...);
rtcSetSharedGeometryBuffer(geom, RTC_BUFFER_TYPE_INDEX, ...);
rtcCommitGeometry(geom);
rtcAttachGeometry(scene, geom);

rtcCommitScene(scene); // ← 50% 时间消耗在此

// ===== cuBQL (GPU) =====
using bvh3f = cuBQL::BinaryBVH<float, 3>;
using box3f = cuBQL::box_t<float, 3>;

// 1. 分配 GPU 内存并上传几何数据
float3 *d_vertices;
uint3 *d_indices;
cudaMalloc(&d_vertices, numVerts * sizeof(float3));
cudaMalloc(&d_indices, numTris * sizeof(uint3));
cudaMemcpy(d_vertices, h_vertices, numVerts * sizeof(float3), cudaMemcpyHostToDevice);
cudaMemcpy(d_indices, h_indices, numTris * sizeof(uint3), cudaMemcpyHostToDevice);

// 2. 计算三角形包围盒 (GPU 内核)
box3f *d_boxes;
cudaMalloc(&d_boxes, numTris * sizeof(box3f));
computeTriangleBounds<<<blocks, threads>>>>(d_boxes, d_vertices, d_indices,
cudaDeviceSynchronize());

// 3. 构建 BVH (GPU)
bvh3f bvh;
cuBQL::BuildConfig cfg;
cfg.makeLeaves = cuBQL::SPATIAL_MEDIAN; // 中等质量
cuBQL::gpuBuilder(bvh, d_boxes, numTris, cfg); // ← GPU 加速构建
cudaDeviceSynchronize();
```

模式 C: 射线相交 (核心转换)

```
// ===== EMBREE (CPU - 单射线循环) =====
for (int i = 0; i < numRays; i++) {
    RTCRayHit rayhit;
    rayhit.ray.org_x = rayOrigins[i].x;
    rayhit.ray.org_y = rayOrigins[i].y;
    rayhit.ray.org_z = rayOrigins[i].z;
    rayhit.ray.dir_x = rayDirs[i].x;
    rayhit.ray.dir_y = rayDirs[i].y;
    rayhit.ray.dir_z = rayDirs[i].z;
    rayhit.ray.tnear = 0.0f;
    rayhit.ray.tfar = 1e20f;
    rayhit.hit.geomID = RTC_INVALID_GEOMETRY_ID;

    rtcIntersect1(scene, &rayhit, NULL); // ← 瓶颈所在!

    if (rayhit.hit.geomID != RTC_INVALID_GEOMETRY_ID) {
        results[i].distance = rayhit.ray.tfar;
        results[i].primID = rayhit.hit.primID;
        results[i].normal = {rayhit.hit.Ng_x, rayhit.hit.Ng_y, rayhit.hit.
    }
}

// ===== cuBQL (GPU - 批量并行内核) =====
__global__ void intersectBatch(
    cuBQL::BinaryBVH<float, 3> bvh,
    const float3* vertices,
    const uint3* indices,
    const float3* rayOrigins,
    const float3* rayDirs,
    int numRays,
    HitResult* results)
{
    int tid = blockIdx.x * blockDim.x + threadIdx.x;
    if (tid >= numRays) return;

    float3 org = rayOrigins[tid];
    float3 dir = rayDirs[tid];

    float closestDist = 1e20f;
    int closestPrim = -1;
    float3 closestNormal = {0, 0, 0};
}
```

```

// 定义相交测试 Lambda (由 cuBQL 遍历框架调用)
auto intersectLambda = [&](int primID) -> float {
    uint3 tri = indices[primID];
    float3 v0 = vertices[tri.x];
    float3 v1 = vertices[tri.y];
    float3 v2 = vertices[tri.z];

    // Möller-Trumbore 射线-三角形相交算法
    float3 e1 = v1 - v0;
    float3 e2 = v2 - v0;
    float3 pvec = cross(dir, e2);
    float det = dot(e1, pvec);

    if (fabsf(det) < 1e-8f) return closestDist;

    float invDet = 1.0f / det;
    float3 tvec = org - v0;
    float u = dot(tvec, pvec) * invDet;
    if (u < 0.0f || u > 1.0f) return closestDist;

    float3 qvec = cross(tvec, e1);
    float v = dot(dir, qvec) * invDet;
    if (v < 0.0f || u + v > 1.0f) return closestDist;

    float t = dot(e2, qvec) * invDet;

    if (t > 0.0f && t < closestDist) {
        closestDist = t;
        closestPrim = primID;
        closestNormal = normalize(cross(e1, e2));
    }

    return closestDist; // 更新搜索半径
};

// 使用 cuBQL 遍历模板遍历 BVH
cuBQL::shrinkingRadiusQuery::forEachPrim(
    intersectLambda,
    bvh,
    org,
    closestDist * closestDist // cuBQL 使用平方距离
);

// 写入结果

```

```

        results[tid].hit = (closestPrim >= 0);
        results[tid].distance = closestDist;
        results[tid].primID = closestPrim;
        results[tid].normal = closestNormal;
    }

    // CPU 端调用
    void traceRays(const bvh3f& bvh,
                  const float3* d_vertices,
                  const uint3* d_indices,
                  const RayBatch& rays,
                  HitResult* d_results)
    {
        int blockSize = 256;
        int numBlocks = (rays.count + blockSize - 1) / blockSize;

        intersectBatch<<<numBlocks, blockSize>>>(
            bvh, d_vertices, d_indices,
            rays.origins, rays.directions,
            rays.count, d_results
        );
        cudaDeviceSynchronize();
    }

```

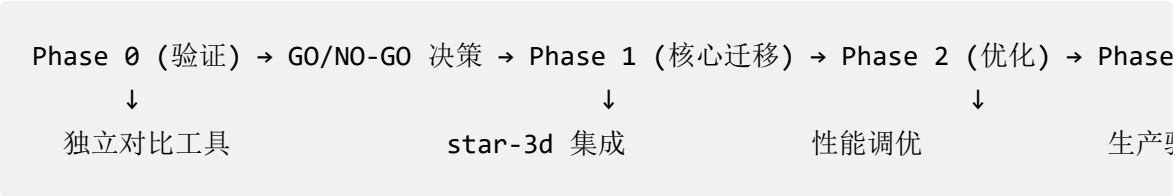
2.3 关键技术差异

方面	Embree	cuBQL	迁移影响
内存模型	CPU（零拷贝共享缓冲区）	GPU（显式上传）	必须复制数据
执行模型	CPU 多线程	GPU 大规模并行	巨大加速潜力
API 风格	面向对象（RTCDevice, RTCScene）	面向数据（BVH 结构体）	更简洁、更显式
批处理	单射线 API	需要批处理	必须重写相交循环
BVH 更新	增量重建	仅支持完全重建	动态场景较慢
自定义几何	C 回调函数	CUDA 设备 lambda	更灵活
错误处理	回调机制	CUDA 错误码	标准 CUDA 模式

三、分阶段实施方案

3.1 总体实施策略

采用渐进式验证方法，每个阶段设置明确的成功标准和退出决策点。



3.2 Phase 0: 独立验证（关键决策点）

目标: 在无 stardis 依赖的独立环境中验证性能增益。

实施方法

1. 创建独立对比工具

- 输入：程序化生成的简化几何（Cornell Box、球体、随机三角形）
- 输出：Embree vs cuBQL 性能对比数据

2. 测试矩阵

场景	三角形数	射线数	用途
Cornell Box	30	1K	最小基准
简单球体	1K	10K	小型场景
复杂网格	10K	100K	中等复杂度
大型网格	100K	1M	压力测试

3. 成功标准

- ☒ BVH 构建加速 $\geq 5\times$
- ☒ 射线相交加速 $\geq 10\times$ （对于批量 $\geq 10K$ 射线）
- ☒ 精度验证：99%+ 射线结果误差 $< 1e-5$
- ☒ CPU-GPU 传输开销 $<$ 总时间的 20%

4. GO/NO-GO 决策点

- **GO**: 相交加速 $\geq 10\times \rightarrow$ 继续 Phase 1
- **NO-GO**: 相交加速 $< 5\times \rightarrow$ 终止迁移，探索替代方案
- **讨论**: $5\times \leq$ 加速 $< 10\times \rightarrow$ 与利益相关者讨论

验证代码结构

```
embree_cubql_comparison/  
├─ CMakeLists.txt
```

```
├─ src/
│   ├── main.cpp           # 驱动程序
│   ├── embree_wrapper.cpp  # Embree BVH + 相交
│   ├── cubql_wrapper.cu    # cuBQL BVH + 相交
│   ├── geometry_generator.cpp # 测试几何生成
│   ├── ray_generator.cpp   # 随机射线生成
│   └── validator.cpp       # 精度对比
├─ include/
│   ├── common_types.h     # 共享数据结构
│   └── timer.h            # 高精度计时
└─ results/
    └── comparison_report.csv # 基准测试结果
```

3.3 Phase 1: 核心迁移

目标: 在 star-3d 中实现 cuBQL 后端，保持 API 兼容性。

迁移范围

组件	文件	迁移内容
设备管理	s3d_device.c/h	替换 RTCDevice 为 CUDA 上下文管理
几何管理	s3d_geometry.c	实现三角形包围盒计算内核
场景管理	s3d_scene_view.c	替换 RTCScene 为 BinaryBVH
射线追踪	s3d_scene_view_trace_ray.c	实现批处理相交内核
自定义几何	s3d_sphere.c	CUDA 球体相交 lambda

替换策略

选项 A: 替换内部头文件（推荐）

创建 cuBQL 版本的内部头文件：

```
// s3d_device_c.h (cuBQL 版本)
struct s3d_device {
    int cudaDeviceID;           // 替代 RTCDevice rtc;
    cudaStream_t stream;        // 异步操作流
    // ... 其他字段保持不变
};

// s3d_geometry.h (cuBQL 版本)
struct geometry {
    cuBQL::box_t<float, 3> bounds; // 替代 RTCGeometry rtc;
```

```

float3 *d_vertices;           // GPU 顶点数据
uint3 *d_indices;            // GPU 索引数据
// ... 其他字段保持不变
};

// s3d_scene_view.c.h (cuBQL 版本)
struct s3d_scene_view {
    cuBQL::BinaryBVH<float, 3> bvh; // 替代 RTCScene rtc_scn;
    cuBQL::box_t<float, 3>* d_boxes; // 替代 RTCGeometry rtc_geom;
    // ... 其他字段保持不变
};

```

选项 B: 条件编译 (向后兼容)

```

// s3d.h 中使用条件包含
#ifdef USE_CUBQL_BACKEND
    #include "gpu/stardis_cubql_geometry.h"
    #include "gpu/stardis_cubql_device.h"
    #include "gpu/stardis_cubql_scene_view.h"
#else
    #include "s3d_geometry.h"
    #include "s3d_device_c.h"
    #include "s3d_scene_view_c.h"
#endif

```

API 兼容性维护

```

// 公共 API (s3d.h) 保持不变
struct s3d_device;           // 不透明指针 - 内部实现改变
struct s3d_scene;            // 不透明指针 - 内部实现改变
struct s3d_scene_view;       // 不透明指针 - 内部实现改变

// 公共函数签名完全保持一致
s3d_device* s3d_device_create(void);
s3d_scene* s3d_scene_create(s3d_device* dev);
void s3d_scene_trace_ray(s3d_scene_view* view,
                        const float org[3],
                        const float dir[3],
                        s3d_hit* hit);

```

3.4 Phase 2: 性能优化

目标: 消除瓶颈，最大化 GPU 利用率。

优化策略

优化项	方法	预期收益
批处理大小	累积射线到阈值（10K-100K）后批量查询	减少内核启动开销
持久化缓冲区	复用 GPU 内存，避免频繁分配/释放	减少内存管理开销
异步传输	使用 CUDA 流重叠 CPU-GPU 传输和计算	隐藏传输延迟
内存对齐	确保数据按 128 字节对齐	提高内存带宽利用率
共享内存缓存	BVH 节点缓存到共享内存	减少全局内存访问

性能剖析工具

- **NVIDIA Nsight Compute:** 内核级性能分析
- **NVIDIA Nsight Systems:** 系统级时间线分析
- **cuda-memcheck:** 内存错误检测

调优目标

- BVH 构建时间 < 20ms（对于 10K 三角形）
- 100 万射线相交时间 < 10ms
- CPU-GPU 传输开销 < 总时间的 10%

3.5 Phase 3: 测试与验证

目标: 确保正确性和生产就绪性。

测试层级

1. 单元测试

- BVH 构建正确性（与 Embree 对比）
- 射线相交精度（逐射线验证）
- 边界情况（空场景、单三角形、退化几何）

2. 集成测试

- 完整 stardis 应用运行
- 与 CPU 版本结果对比（逐像素验证）
- 多场景回归测试

3. 性能基准测试

- 标准场景性能对比
- 不同批处理大小的影响

- 内存消耗分析

验证标准

- ✅ 功能正确性: 100% 测试用例通过
- ✅ 数值精度: 99.9%+ 结果误差 < 1e-5
- ✅ 性能目标: 整体应用加速 ≥ 1.5x
- ✅ 内存稳定性: 无泄漏, 长时间运行稳定

四、风险评估与缓解策略

4.1 技术风险

风险	概率	影响	缓解措施
CPU-GPU 传输开销占主导	中	高	批处理优化 (批量 > 10K 射线)、持久化缓冲区、异步流
精度不匹配导致科学结果失效	低	高	使用双精度 BinaryBVH<double, 3>、几何缩放、严格验证框架
2x 加速不足以证明迁移成本	中	中	Phase 0 提供早期退出点, 避免浪费资源
cuBQL API 变更破坏代码	低	中	固定到特定 cuBQL 提交版本
大型场景内存耗尽	低	中	分块处理、流式 BVH、压缩技术

4.2 实施风险

风险	概率	影响	缓解措施
与 star-3d 集成问题	中	中	早期 API 模拟、增量测试
GPU 内核调试时间过长	高	中	使用 cuda-gdb、CUDA-MEMCHECK、Nsight
性能调优陷入兔子洞	中	低	设定性能目标, 达到即停止

4.3 致命场景与应对

场景	概率	应对措施
Phase 0 加速 < 2x	10%	终止迁移, 考虑替代方案 (OptiX、混合 CPU-GPU)
精度误差 > 1e-3	5%	切换到双精度、几何缩放、数值稳定性优化
GPU 内存不足	5%	实现流式/分块处理

五、完整迁移检查清单

5.1 必需修改

☐ 替换 Embree 头文件引用

```
// 旧
#include <embree4/rtcore.h>

// 新
#include <cuBQL/bvh.h>
#include <cuBQL/builder/cuda.h>
#include <cuBQL/traversal/shrinkingRadiusQuery.h>
```

☐ 修改数据结构

```
// 旧
RTCDevice device;
RTCScene scene;
RTCGeometry geom;

// 新
cuBQL::BinaryBVH<float, 3> bvh;
cuBQL::box_t<float, 3>* d_boxes;
```

☐ 分配 GPU 内存

```
cudaMalloc(&d_vertices, ...);
cudaMalloc(&d_indices, ...);
cudaMalloc(&d_boxes, ...);
```

☐ 实现包围盒计算内核

```
__global__ void computeTriangleBounds(...) { ... }
```

☐ 替换 rtcCommitScene 为 gpuBuilder

```
// 旧
rtcCommitScene(scene);
```

```
// 新
cuBQL::gpuBuilder(bvh, d_boxes, numPrimitives, cfg);
```

- ☐ 替换 `rtcIntersect1` 为批处理内核

```
__global__ void intersectBatch(...) {
    cuBQL::shrinkingRadiusQuery::forEachPrim(lambda, bvh, org, maxDis
}
```

5.2 可选优化

- ☐ 使用持久化 GPU 缓冲区（避免重复分配/释放）
- ☐ 批量射线上传（摊销传输开销）
- ☐ 异步流（重叠计算和传输）
- ☐ 自定义内存分配器（`GpuMemoryResource`）
- ☐ 性能剖析（`nvprof`/`Nsight` 识别瓶颈）

5.3 测试验证

- ☐ **功能测试:** 与 Embree 结果一致（ $< 1e-5$ 误差）
- ☐ **性能测试:** 测量实际加速比
- ☐ **内存测试:** 检查泄漏，验证清理
- ☐ **边缘情况:** 空场景、单三角形、退化几何

六、决策建议

6.1 技术可行性评估

评估维度	状态	依据
API 映射完整性	✅ 已确认	所有 Embree 调用均有 cuBQL 等效方案
环境可行性	✅ 已确认	RTX 3060+ 可用，CUDA 12+ 支持
范围可控性	✅ 已确认	隔离在 star-3d， $< 3K$ LOC
依赖隔离	✅ 已验证	外部模块无需修改
精度可达性	⚠️ 待验证	Phase 0 需验证 $< 1e-5$ 误差
性能目标	⚠️ 待验证	Phase 0 需验证 $\geq 10x$ 加速

6.2 推荐行动

结论: ✅ 有条件实施 - 先执行 **Phase 0** 验证

理由:

1.  技术可行性已确认 (cuBQL 支持所需功能)
2.  API 映射完整且可操作
3.  低风险验证路径存在 (独立工具)
4.  性能必须在全面投入前验证
5.  2x 整体加速是上限 (Amdahl 定律)

必需行动: 实施 Phase 0 验证

决策点: Phase 0 结果后:

- 若相交加速 $\geq 10x$ → **继续 Phase 1**
- 若相交加速 $< 5x$ → **终止**, 探索替代方案
- 若 $5x \leq$ 相交加速 $< 10x$ → **与利益相关者讨论**

6.3 替代方案 (若 Phase 0 失败)

方案 A: NVIDIA OptiX

- **优点:** 成熟、RT Core 加速、广泛文档
- **缺点:** 更复杂的 API、许可考虑、仅 Windows/Linux

方案 B: 混合 CPU-GPU

- **优点:** 增量迁移、同时利用 Embree (细粒度) 和 cuBQL (粗粒度)
- **缺点:** 复杂编排、有限加速

方案 C: 保持 Embree + CPU 优化

- **优点:** 无迁移风险、已知数量
- **缺点:** 错失 GPU 加速机会

七、总结

关键点

1. **问题明确:** 50% 时间消耗在 Embree 射线追踪, 瓶颈清晰
2. **方案可行:** cuBQL 提供所需功能, API 映射完整
3. **范围可控:** 仅影响 star-3d 库, 外部模块隔离
4. **风险可管理:** 分阶段方法提供早期验证和退出点
5. **收益现实:** 保守估计 2x 整体加速 (受 Amdahl 定律限制)

成功因素

-  **清晰的性能瓶颈** (单一入口点消耗 50% 时间)
-  **成熟的 GPU 库** (cuBQL 稳定可用)
-  **低风险验证路径** (独立工具先行)
-  **依赖完全隔离** (外部无影响)

实施路线图



最终建议

立即启动 Phase 0 验证。这是低成本、高价值的决策输入，将在 1-2 周内提供明确的数据支持后续决策。无论 Phase 0 结果如何，都将为项目提供宝贵的技术洞察。

文档版本: 1.0
创建日期: 2026-01-23
状态: ☒ 可行性已确认 - 建议实施 Phase 0 验证
下一步: 实施独立性能对比工具，执行基准测试

附录：参考文档

文档	用途
migration_strategy_executive_summary.md	战略层面概述
embree_to_cubql_api_migration_mapping.md	完整 API 转换指南
embree_cubql_performance_comparison_plan.md	Phase 0 实施细节
embree_dependency_scope_analysis_CRITICAL_UPDATE.md	依赖范围分析
embree_exposed_type_access.md	外部访问验证
embree_couple.md	Embree 调用流程分析
environment_requirements.md	环境配置需求